En la primera parte se observa el servidor json-server levantado correctamente en el puerto 3000, exponiendo los endpoints /productos, /categorias y /perfil. Esto confirma que el backend simulado está activo y listo para recibir solicitudes HTTP.

En la segunda parte se visualiza el contenido del archivo db.json, donde se encuentran los datos iniciales del sistema, incluyendo productos con sus atributos (id, nombre, precio y categoría). Esto demuestra que el servidor está sirviendo datos estructurados correctamente en formato JSON.

Finalmente, en la tercera imagen se muestra la ejecución del cliente asíncrono cliente_async_ecomarket.py. El resultado del dashboard indica que las tres fuentes (productos, categorías y perfil) fueron consultadas de manera concurrente y respondieron exitosamente, sin errores. Además, el tiempo total de ejecución (0.015 segundos) evidencia que las peticiones se realizaron en paralelo y no de forma secuencial.

```
christianortiz@192 Semana3 % json-server --watch db.json --port 3000
--watch/-w can be omitted, JSON Server 1+ watches for file changes by de
fault
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...

(^_^)

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/productos
http://localhost:3000/categorias
http://localhost:3000/perfil
```


```
[
  {
    "id": "1",
    "nombre": "Laptop",
    "precio": 15000,
    "categoria": "tecnologia"
  },
  {
    "id": "2",
    "nombre": "AudÃfonos",
    "precio": 800,
    "categoria": "tecnologia"
  }
]
```


```
christianortiz@192 Semana3 % python3 cliente_async_ecomarket.py

Resultado del dashboard:
{'datos': {'productos': [{'id': '1', 'nombre': 'Laptop', 'precio': 15000, 'categoria': 'tecnologia'}, {'id': '2',
nombre': 'Audífonos', 'precio': 800, 'categoria': 'tecnologia'}], 'categorias': [{'id': '1', 'nombre': 'tecnologi
}, {'id': '2', 'nombre': 'hogar'}], 'perfil': {'id': 1, 'nombre': 'Christian', 'rol': 'cliente'}}, 'errores': {}}
Tiempo total: 0.015 segundos
```

Reto 6 - Crítico de Decisiones de Concurrencia

ADR-1 — Estrategia de ejecución concurrente

Contexto

El dashboard necesita realizar varias peticiones HTTP en paralelo para reducir el tiempo total de carga.

Decisión

Se utiliza asyncio.gather(*tareas, return_exceptions=True) para ejecutar las peticiones concurrentemente y recolectar todos los resultados al finalizar.

Alternativas

- asyncio.as_completed() para procesar resultados conforme llegan.
- asyncio.wait() para mayor control sobre cancelaciones.
- Ejecución secuencial.

Consecuencias

- Implementación simple y clara.
- Reduce el tiempo total respecto a ejecución secuencial.
  − No permite procesamiento progresivo.
  − El tiempo total depende de la petición más lenta.

ADR - 2 - Uso de Client Session compartida

Contexto

El dashboard realiza múltiples solicitudes al mismo servidor dentro del mismo flujo de ejecución.

Decisión

Se utiliza una única instancia de aiohttp.ClientSession compartida para todas las peticiones.

Alternativas

- Crear una sesión por cada petición.
- Implementar un pool manual de sesiones.

Consecuencias

- Reutilización de conexiones (connection pooling).
- Mejor rendimiento y menor overhead.
  − Menor flexibilidad si se requieren configuraciones muy distintas por petición.
  − Riesgo si se modifican configuraciones globales de la sesión.

ADR- 3 - Control de concurrencia con semáforo

Contexto

Se ejecutan múltiples peticiones simultáneas al crear varios productos.

Decisión

Se limita la concurrencia mediante asyncio.Semaphore(10).

Alternativas

- No limitar concurrencia.
- Límite configurable por entorno.
- Concurrencia adaptativa.

Consecuencias

- Previene saturación del servidor y del cliente.
- Controla el consumo de recursos.

RETO 8 - Diseñador de Suite de Pruebas Asíncronas

Reporte de bugs encontrados durante testing y correcciones aplicadas.

```
============================================ test session starts ============================================
platform darwin -- Python 3.9.6, pytest-8.4.2, pluggy-1.6.0 -- /Library/Developer/CommandLineTools/usr/bin/python3
cachedir: .pytest_cache
rootdir: /Users/christianortiz/Desktop/Semanas/Semana3
plugins: asyncio-1.2.0
asyncio: mode=strict, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=functio
n
collected 22 items

test_cliente_async.py::test_listar_productos_equivalencia PASSED                                          [  4%]
test_cliente_async.py::test_obtener_producto_equivalencia PASSED                                          [  9%]
test_cliente_async.py::test_crear_producto_equivalencia PASSED                                            [ 13%]
test_cliente_async.py::test_error_http_manejado PASSED                                                    [ 18%]
test_cliente_async.py::test_eliminar_producto_ok PASSED                                                   [ 22%]
test_cliente_async.py::test_gather_tres_exitos PASSED                                                     [ 27%]
test_cliente_async.py::test_gather_un_fallo_return_exceptions PASSED                                      [ 31%]
test_cliente_async.py::test_gather_sin_return_exceptions_propagacion PASSED                               [ 36%]
test_cliente_async.py::test_dashboard_falla_una_fuente PASSED                                             [ 40%]
test_cliente_async.py::test_semaforo_limita_concurrencia PASSED                                           [ 45%]
test_cliente_async.py::test_timeout_individual PASSED                                                     [ 50%]
test_cliente_async.py::test_cancelacion_en_cadena PASSED                                                  [ 54%]
test_cliente_async.py::test_timeout_global_dashboard PASSED                                               [ 59%]
test_cliente_async.py::test_cancelled_error_no_resource_leak PASSED                                       [ 63%]
test_cliente_async.py::test_peticion_cancelada_no_error_log PASSED                                        [ 68%]
test_cliente_async.py::test_todas_fallan_simultaneamente PASSED                                           [ 72%]
test_cliente_async.py::test_servidor_cierra_conexion PASSED                                               [ 77%]
test_cliente_async.py::test_respuesta_despues_timeout PASSED                                              [ 81%]
test_cliente_async.py::test_dos_peticiones_mismo_endpoint_distintos_params PASSED                         [ 86%]
test_cliente_async.py::test_sesion_cierra_con_errores PASSED                                              [ 90%]
test_cliente_async.py::test_gather_orden_preservado PASSED                                                [ 95%]
test_cliente_async.py::test_multiple_creaciones_concurrentes PASSED                                       [100%]

============================================= 22 passed in 0.54s ============================================
```